线段树

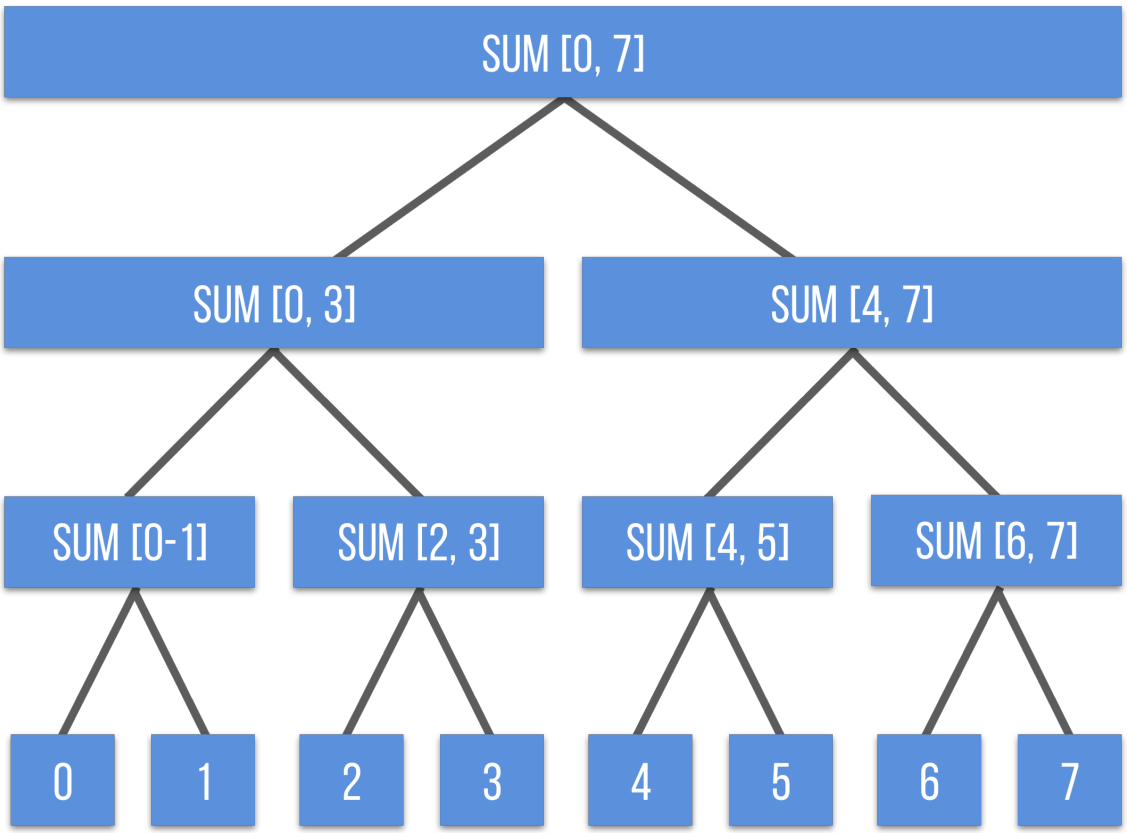
1 定义
2 性质
3 存储方式
4 建立线段树
4.1 向上传递信息操作 （Push Up）
4.2 建树操作（Build）
5 功能
5.1 区间求和
5.2 单点修改
5.3 区间最值
5.4 区间修改
5.4.1 延迟标记
5.4.2 Push Down 操作
5.4.3 区间查询
5.4.4 单点修改
6 复杂度分析
7 总结
8 例题
8.1 区间最大公约数
9 定义
10 功能
10.1 插入与删除操作
10.2 查询第 k 大元素
10.3 查询 x 的排名
10.4 查询前驱
10.5 查询后继
11 动态开点线段树
11.1 定义
11.2 操作
11.2.1 单点修改
11.2.2 区间查询
11.3 复杂度分析

1 定义

引入一个单点修改，区间查询问题：

> 给定数列 $a[1], a[2], \ldots, a[n]$，你需要依次进行 q 个操作，操作有两类：
>
> - `1 i x`：给定 i, x，将 $a[i]$ 加上 x；
> - `2 l r`：给定 l, r，求 $\sum_{i=l}^{r} a[i]$ 的值（换言之，求 $a[l] + a[l+1] + \cdots + a[r]$ 的值）。
>
> 对于所有数据，$1 \le n, q \le 10^6$, $|a[i]| \le 10^6$, $1 \le l \le r \le n$, $|x| \le 10^6$。

线段树是如何表示的：



将整个数组区段，**通过二分区段的方式，将每一个区段的答案记录在二叉树每一个节点上。** 当查询的时候只需要在这个二叉树中对边界做判断，然后拆分成子区间即可。

所以你应该可以理解，为什么它要叫“线段树”了吧，答案就是它会将整个区段抽象成一个大“线段”，然后根据表示的范围，二分划分成子“线段”。

如此给出线段树的定义：**线段树是一棵二叉树，线段树上每个结点对应的是序列的一段区间，每一个叶子结点对应的是序列的一个元素。树上的每个节点都维护一个区间，根维护的是整个区间，每个节点维护的是父亲的区间二等分后的其中一个子区间。** 当有 a 个元素时，对区间的操作可以在 $O(\log n)$ 复杂度时间内完成。另外，从二叉树结构上，可以看出线段树是一棵完美二叉树（Perfect Binary Tree），即所有叶子的深度都相同，并且每个节点要么是叶子，要么有 2 个子树。

2 性质

1. 线段树的每个节点都代表一个区间。
2. 线段树具有唯一的根节点，代表的区间是整个统计范围，如$[1, N]$。
3. 线段树的每个叶节点都代表一个长度为 1 的区间$[x, x]$。
4. 对于每个内部节点$[l, r]$，它的左子节点是$[l, mid]$, 右子节点是$[mid + 1, r]$ 其中$mid = \lfloor \frac{l+r}{2} \rfloor$
5. 若线段树去掉最后一层，一定是一个满二叉树。

3 存储方式

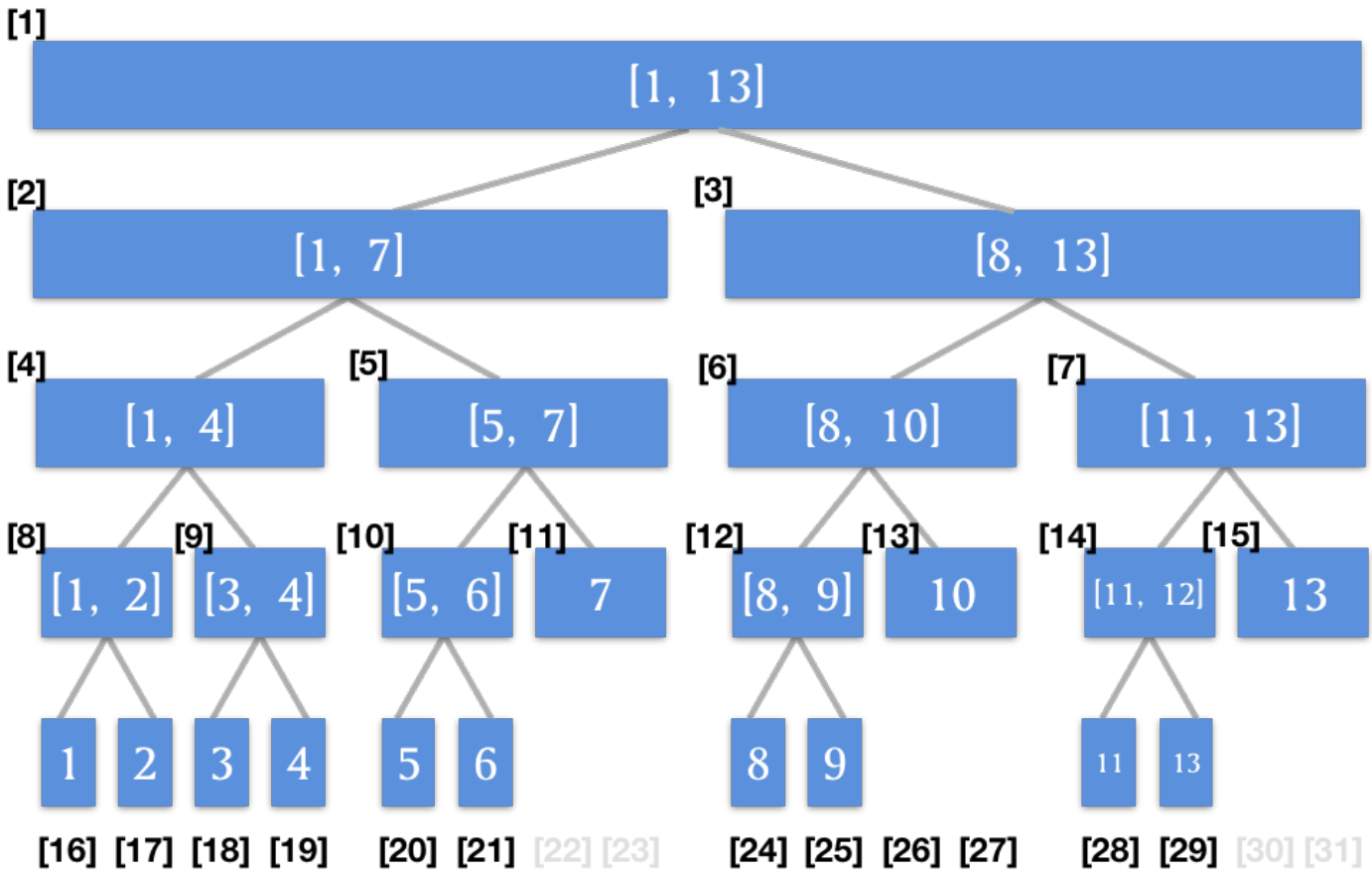
按照二叉树编号方法对线段树进行编号

根节点编号为 1，编号为 x 的节点的左子节点编号为 $x * 2$, 右子节点编号为 $x * 2 + 1$。

可以使用**二叉树的顺序存储结构**来存储线段树。

线段树是一棵 **完美二叉树(Perfect Binary Tree)**，可是题目中给出的结点个数不一定是 2^N 次幂个。所以，这就带来了 **空间结构退化的问题**。

这里我们假设 `N = 13` 这个情况，然后我们通过之前的线段树代码进行代码实现后其结构变成了这样：



通过上图，我们发现如果我们使用 `2N = 26` 的数组空间，实际上线段树已经覆盖了下标 `31` ，这个场景下我们开 `2N` 的数组是不够的。

根据线段树结构的性质，会把这 N 个点当做叶子节点来看待。由于线段树是一棵满二叉树，假设它有 H 层，第 H 层节点数是 2^{H-1} 个，一共 $2^H - 1$ 个节点。假设共 S 个节点，来推导一下：

$$S = 2^H - 1 = 2^{H-1} \times 2 - 1 = 2N - 1 \tag{1}$$

可以发现树的最后层节点在数组中保存的位置不是连续的，直接空出数组中多余的位置即可。即最后一层至少保证有$2N$个节点。所以**保存线段树的数组长度要不小于 $4N$ 才能保证不会越界**。

4 建立线段树

将线段树使用以下方式来声明。

```
const int maxn = 1e4 + 7;
int tree[maxn << 2];
```

使用数组 `tree[N]` 来描述二叉树，对于节点 `tree[i]` ，它的左右孩子节点分别是 `tree[2 * i]` 和 `tree[2 * i + 1]` 。在代码中为了稍微稍微做一些加速，所以用 `tree[i << 2]` 和 `tree[i << 2 | 1]` 来表示。

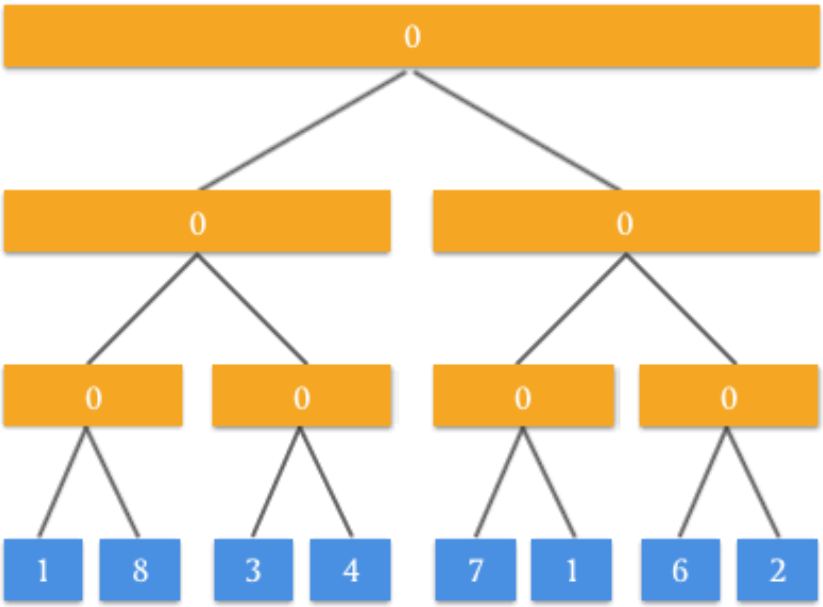
4.1 向上传递信息操作（Push Up）

需要构造（Build）一棵线段树，与之同时还要考虑这棵线段树的 Push Up 操作。这个 Push Up 操作可以理解成是一种**向上更新**的概念。

```
void push_up(int rt) {
    tree[rt] = tree[rt << 1] + tree[rt << 1 | 1];
}
```

通过代码可以看出，其实是使用下方的两个孩子节点，来更新其共同的父亲结点。从二叉树的角度来看，这个操作由于是向上方来更新维护的数据，所以称之为 **Push Up 操作** 。

具体的 Push Up 操作是为了实现什么效果呢？看这张动图你就懂了：



4.2 建树操作（Build）

有了 Push Up 操作后，就可以自底向上来构建这棵二叉树了。由于知道根节点是 `tree[1]` ，而所有需要 Push Up 操作的元素都是最底部的叶子结点，所以在构造的时候需要先访问到最底部，递归调用 Push Up 向上更新即可。

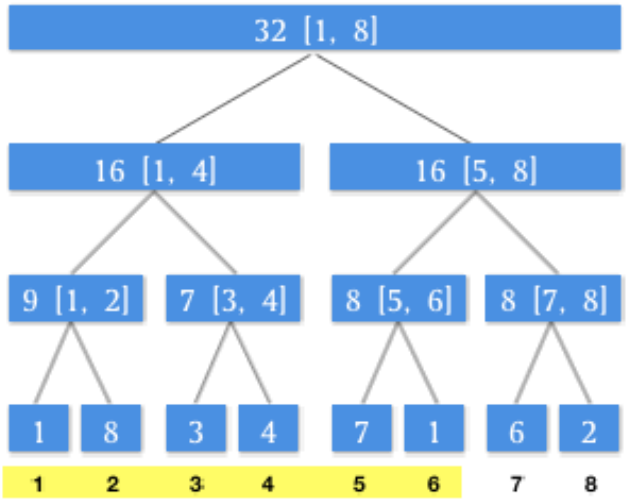
```
/*
l 当前节点描述范围的左边界
r 当前节点描述范围的右边界
rt 代表下标，tree[rt] 代表当前节点
*/
void build(int l, int r, int rt) {
    if (l == r) {
        tree[rt] = num[++cur];
        return ;
    }
    int m = (l + r) >> 1;
    build(l, m, rt << 1);
    build(m + 1, r, rt << 1 | 1);
    push_up(rt);
}
```

从这里可以看的出，其实构造一棵线段树，就需要执行多达 $O(n\log n)$ 的复杂度。

5 功能

5.1 区间求和

一个线段树，线段树每一个节点代表的是一个区间。如果查询 $[1,6]$ 这个区间，而当前结点是根节点 $[1,8]$ ，左区间是 $[1,4]$ 右区间是 $[5,8]$ 这种情况，没有一个恰好能满足查询区域的区间范围。这时候只要把 $[1,8]$ 划分成 $[1,4]$ 和 $[5,6]$ 分头查找就可以了。用树的遍历思想，其实就是两次递归可以了。




```
/*
L 待查询区间左边界
R 待查询区间右边界
l 当前节点描述范围的左边界
r 当前节点描述范围的右边界
```

```
rt 代表下标，tree[rt] 代表当前节点
 */
int query(int L, int R, int l, int r, int rt) {
    if (L <= l && r <= R) {
        return tree[rt];
    }
    int m = (l + r) >> 1;
    int ret = 0;
    if (L <= m) ret += query(L, R, l, m, rt << 1);
    if (R > m)  ret += query(L, R, m + 1, r, rt << 1 | 1);
    return ret;
}
```

5.2 单点修改

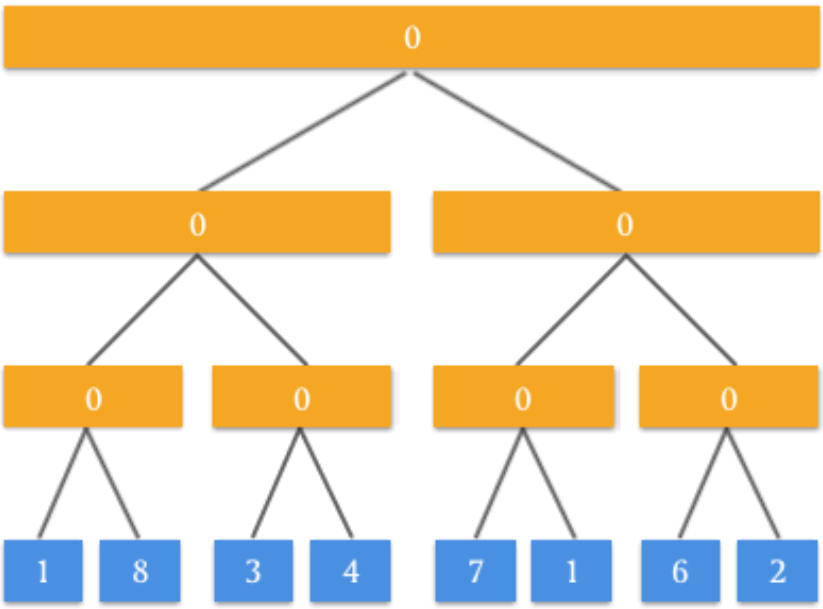
这一点是 ST 算法不具备的能力。其实单点更新操作与 `build` 方法的想法十分相似，只要递归到叶子节点，将更新值修改掉，之后逐渐向上执行 `push_up` 更新，其实就完成了新状态的维护。

```
/*
p 待更新的下标，指的在 num 中的下标
replace 待替换的值
l 当前节点描述范围的左边界
r 当前节点描述范围的右边界
rt 代表下标，tree[rt] 代表当前节点
 */
void update(int p, int replace, int l, int r, int rt) {
    if (l == r) {
        tree[rt] = replace;
        return ;
    }
    int m = (l + r) >> 1;
    if (p <= m) update(p, replace, l, m, rt << 1);
    else update(p, replace, m + 1, r, rt << 1 | 1);
    push_up(rt);
}
```

5.3 区间最值

上面通过 Push Up 方法中，父节点等于两个孩子结点之和来构造的区间和树。通过这个思路来想，如果修改 Push Up 操作，**将父亲节点等于两个孩子节点的最大值，就完成了区间最大线段树**。

```
void push_up(int rt) {
    tree[rt] = max(tree[rt << 1], tree[rt << 1 | 1]);
}
```




```
int query(int L, int R, int l, int r, int rt) {
    if (L <= l && r <= R) {
        return tree[rt];
    }
    int m = (l + r) >> 1;
    int ret = INT_MIN;
    if (L <= m)
        ret = max(ret, query(L, R, l, m, rt << 1));
    if (R > m)
        ret = max(ret, query(L, R, m + 1, r, rt << 1 | 1));
    return ret;
}
```

5.4 区间修改

在“区间修改”指令中，如果某个节点被修改区间$[l,r]$完全覆盖，那么以该节点为根的整棵子树中的所有节点存储的信息都需要发生变化，若逐一进行更新，将使得一次区间修改指令的时间复杂度增加到 $O(N)$。

要确定一个目标，我们的**节点更新其实是为了查询到最新的结果**。

如果在一次修改指令中发现节点 p 代表的区间$[p_l,p_r]$修改区间 $[l,r]$完全覆盖，并且逐一更新了子树 p 中的所有节点，但是在之后的查询指令中却根本没有用到 $[l,r]$ 的子区间作为候选答案，那么更新 P 的整棵子树就是无用的。

5.4.1 延迟标记

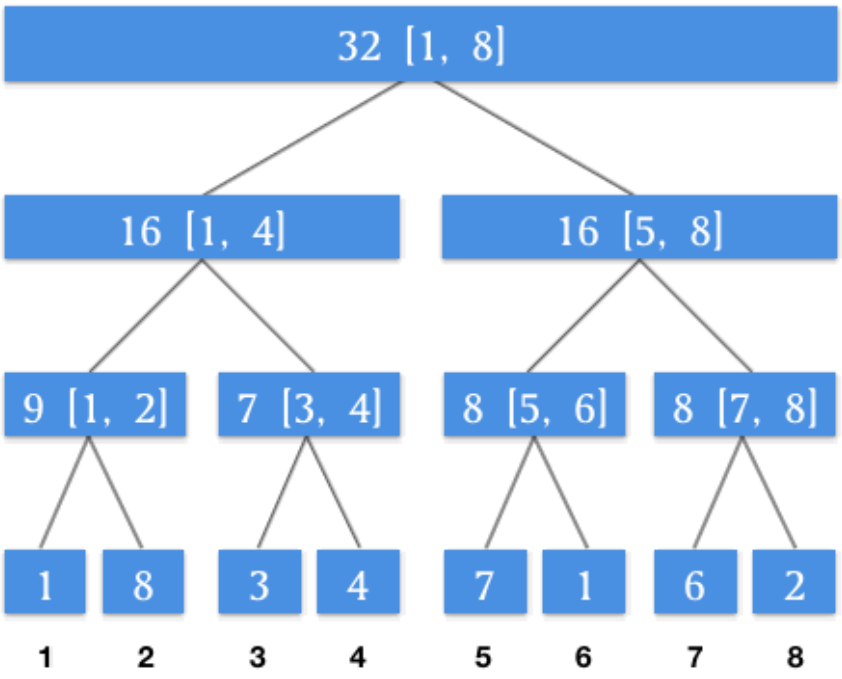
我们在执行修改指令时，只修改到满足$l \le p_l \le p_r \le r$的节点后**停止递归修改**，只不过在回溯之前向节点 p 增加一个标记，标识**“该节点曾经被修改，但其子节点尚未被更新”。**该标记的作用如下：

1.若多个修改指令都可以满足同一个节点p，$l \le p_l \le p_r \le r$成立，那么可以将这多次的修改影响值积累在p节点的标记中。

2.若在后续某个指令中，节点p不能够满足$l \le p_l \le p_r \le r$，需要从节点 p 向下递归，我们再检查 p 是否具有标记。若有标记，就根据标记信息更新 p 的两个子节点，同时为 p 的两个子节点增加标记，然后清除 p 的标记。

也就是说，对任意节点的修改都延迟到“**在后续操作中递归进入它的父节点时”再执行**。这样一来，每条查询或修改指令的时间复杂度都降低到了$(\log N)$。这些标记被称为“**延迟标记”**。延迟标记提供了线段树中**从上往下传递信息的方式**。这种“延迟”也是设计算法与解决问题的个重要思路。

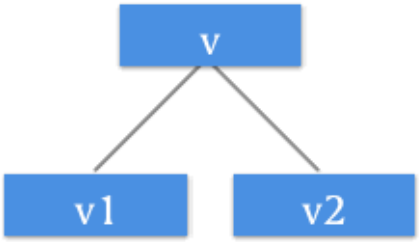
例如对线段树进行一个区间更新的操作，将 `[3, 6]` 这个区间上的所有数字增加 `4` 。此时我们的增量数组如图所示：



5.4.2 Push Down 操作

所谓 Push Down 操作，就是向下传递延迟标记过程。

```
//rt为节点编号，m是rt节点表示的区间长度
void push_down(int rt, int m) {
    if (target[rt]) {
        int l = rt << 1, r = rt << 1 | 1;
        target[l] += target[rt];
        target[r] += target[rt];
        tree[l] += target[rt] * (m - (m >> 1));
        tree[r] += target[rt] * (m >> 1);
        target[rt] = 0;
    }
}
```



5.4.3 区间查询

```
int query(int L, int R, int l, int r, int rt) {
    if (L <= l && r <= R) {
        return tree[rt];
    }
    push_down(rt , r - l + 1);
    int m = (l + r) >> 1;
    int ret = 0;
    if (L <= m) ret += query(L , R , l, m, rt << 1);
    if (m < R) ret += query(L , R , m + 1, r, rt << 1 | 1);
    return ret;
}
```

5.4.4 单点修改

```
void update(int L, int R, int c, int l, int r, int rt) {
    if (L <= l && r <= R) {
        target[rt] += c;
        tree[rt] += c * (r - l + 1);
        return;
    }
    push_down(rt, r - l + 1);
    int m = (l + r) >> 1;
    if (L <= m) update(L, R, c, l, m, rt << 1);
    if (m < R) update(L, R, c, m + 1, r, rt << 1 | 1);
    push_up(rt);
}
```

6 复杂度分析

无论是区间求和，还是 RMQ，通过线段树的数据结构来描述数据，实现了构造 $O(4n)$ ，查询 $O(k\log n)$ (k 代表可能出现的跨区域多路径)，单点更新 $O(\log n)$ 。

7 总结

通过 RMQ 线段树通过区间和线段树的简单转换，其实已经知道了，线段树能解决的问题不仅仅是这两个场景。其实只要是区间查询、区间操作问题中，并且区间内有确定运算结果场景下，都可以通过线段树来优化查询、更新的复杂度，例如：区间求 GCD（最大公约数）、区间求异或结果等等。

8 例题

8.1 区间最大公约数

给定一个长度为 N 的数列 A，以及 M 条指令，每条指令可能是以下两种之一：

1. `C l r d`，表示把 $A[l], A[l+1], \ldots, A[r]$ 都加上 d。
2. `Q l r`，表示询问 $A[l], A[l+1], \ldots, A[r]$ 的最大公约数(GCD)。

对于每个询问，输出一个整数表示答案。

思路

如果没有修改操作只有查询操作，那么直接维护区间的最大公约数就可以了。

如果只有单点修改，一样只需要维护区间的最大公约数就可以了。

如果是区间修改那么只维护区间最大公约数就不可以了，因为如果给某个区间都加上x，那么可以发现$\gcd(a,b,c)$与$\gcd(a+x,b+x,c+x)$没有什么关系，只根据之前的最大公约数是求不出来加上某个数后的最大公约数的。

最大公约数有这样一个性质：$\gcd(a_1,a_2,\ldots,a_n)=\gcd(a_1,a_2-a_1,\ldots,a_n-a_{n-1})$

这里简单证明一下，根据性质

$$\begin{aligned}\gcd(a,b)&=\gcd(b,a)\\ \gcd(a,b)&=\gcd(a,b-a)\\ \gcd(a,b,c)&=\gcd(gcd(a,b),c)\end{aligned}\tag{2}$$

有

$$\begin{gathered}\gcd(a,b,c,d)=\gcd(a,b-a,c,d)= \\ gcd(a,b-a,c-b+a,d)=\gcd(a,b-a,c-b,d)= \\ \gcd(a,b-a,c-b,d-c+b)=\gcd(a,b-a,c-b,d-c+a)= \\ \gcd(a,b-a,c-b,d-c)\end{gathered} \tag{3}$$

可以归纳证明多个变量的情况。

因此可以用线段树来维护一个差分序列，这样区间修改就可以变成单点修改了，在维护区间最大公约数的同时再维护一个区间和。

当查询区间$[l,r]$，就是求$\gcd(a_l,a_{l+1}-a_l,\ldots,a_r-a_{r-1})$，$a_l$可以用维护的区间和得到，$\gcd(a_{l+1}-a_l,\ldots,a_r-a_{r-1})$可以用维护的区间最大公约数得到。

```cpp
#include <bits/stdc++.h>
using namespace std;

typedef long long LL;

const int N = 5e5 + 10;

LL a[N];
struct Node {
    int l, r;
    LL s, d;
} tr[N * 4];

LL gcd(LL a, LL b) {
    return b ? gcd(b, a % b) : a;
}

void build(int u, int l, int r) {
    if (l == r) {
        tr[u] = {l, r, a[l] - a[l - 1], a[l] - a[l - 1]};    // 维护的是差分序列
    } else {
        int mid = l + r >> 1;
        build(u << 1, l, mid);
        build(u << 1 | 1, mid + 1, r);
        tr[u] = {l, r, tr[u << 1].s + tr[u << 1 | 1].s, gcd(tr[u << 1].d, tr[u << 1 | 1].d)};
    }
}

void modify(int u, int x, LL c) {
    if (tr[u].l == x && tr[u].r == x) {
        tr[u].s += c, tr[u].d += c;
    } else {
        int mid = tr[u].l + tr[u].r >> 1;
        if (x <= mid) modify(u << 1, x, c);
        else modify(u << 1 | 1, x, c);
        tr[u].s = tr[u << 1].s + tr[u << 1 | 1].s;
```

```
        tr[u].d = gcd(tr[u << 1].d, tr[u << 1 | 1].d);
    }
}

Node query(int u, int l, int r) {
    if (tr[u].l >= l && tr[u].r <= r) return tr[u];
    int mid = tr[u].l + tr[u].r >> 1;
    if (r <= mid) {
        return query(u << 1, l, r);
    } else if (l >= mid + 1) {
        return query(u << 1 | 1, l, r);
    } else {
        Node t1 = query(u << 1, l, r), t2 = query(u << 1 | 1, l, r);
        return {l, r, t1.s + t2.s, gcd(t1.d, t2.d)};
    }
}

int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    for (int i = 1; i <= n; i++) {
        scanf("%lld", a + i);
    }
    build(1, 1, n + 1);
    while (m--) {
        char op[5];
        int l, r;
        scanf("%s %d %d", op, &l, &r);
        if (op[0] == 'C') {
            LL c;
            scanf("%lld", &c);
            modify(1, l, c), modify(1, r + 1, -c);
        } else {
            LL d = abs(query(1, 1, l).s);    // a[l]通过差分序列的前缀和求得
            if (l + 1 <= r) printf("%lld\n", abs(gcd(d, query(1, l + 1, r).d)));
    // 当l+1 <= r才存在剩余部分的最大公约数
            else printf("%lld\n", d);
        }
    }

    return 0;
}
```

权值线段树

1 定义

权值线段树较普通线段树不同地方在于：权值线段树的数组范围

`n`是它的值域大小，储存的内容是这个值的出现次数。

2 功能

由于权值线段树的特性，因此权值线段树可以用来实现一些普通平衡树的功能，如：

1.插入某个数

2.删除某个数

3.求某个数`x`的排名

4.求排名为`x`的数值

5.求某个数的前驱

6.求某个数的后继

但权值线段树也有一些弊端，因为权值线段树的数组大小是值域大小，因此当值域非常大时（如10^9），需要进行离散化，变成离线数据结构。

2.1 插入与删除操作

权值线段树的插入与删除操作与普通线段树一致，是一种单点修改操作：

```
void update(int root, int left, int right, int num, int v) {  //增加v个值为num的数字
  int lc = root << 1, rc = root << 1 | 1, mid = (left + right) >> 1;
  if (left == right) {
    tree[root] += v;  //统计出现次数
    return;
  }
  if (num <= mid) update(lc, left, mid, num, v);
  else update(rc, mid + 1, right, num, v);
  tree[root] = tree[lc] + tree[rc];
}

```

2.2 查询第 k 大元素

当左子树内权值数量大于等于剩余查询个数，则递归进入左子树进行查询，否则递归进入右子树进行查询。

```
int queryK(int root, int left, int right, int num) {
  int lc = root << 1, rc = root << 1 | 1;
  int mid = (left + right) >> 1;
  if (left == right) return left;
  if (tree[lc] >= num) return queryK(lc, left, mid, num);
  else return queryK(rc, mid + 1, right, num - tree[lc]);
}
```

2.3 查询 x 的排名

权值线段树查询 x 的排名

```
int query(int root, int left, int right, int qleft, int qright) {
  int lc = root << 1, rc = root << 1 | 1, mid = (left + right) >> 1;
  if (qleft <= left && right <= qright) return tree[root];
  int ans = 0;
  if (qleft <= mid) ans += query(lc, left, mid, qleft, qright);
  if (qright > mid) ans += query(rc, mid + 1, right, qleft, qright);
  return ans;
}
int rank(int x) {
  if (x <= 1) return 1;
  return query(1, 1, SIZE, 1, x - 1) + 1;
}
```

2.4 查询前驱

如果 `num` 在当前区间的左半边那么答案一定在左区间，于是递归到左区间寻找，否则优先在右区间进行寻找，如果找到答案的话就返回，否则再在左区间寻找。

```
int getpre(int root, int left, int right, int num) {//寻找num的前驱
  if (!tree[root]) return 0;//该区间为无前驱
  if (left == right) return left;
  int lc = root << 1, rc = root << 1 | 1, mid = (left + right) >> 1;
  if (num <= mid) return getpre(lc, left, mid, num);//前驱一定在左子树上
  int t = getpre(rc, mid + 1, right, num);//前驱不一定在左子树上，尝试在右子树上找
  if (t) return t;//在右子树上找到
  return getpre(lc, left, mid, num);//在右子树上没有找到，再尝试在左子树上找
}
```

2.5 查询后继

如果 `num` 在当前区间的左半边那么答案一定在左区间，于是递归到左区间寻找，否则优先在右区间进行寻找，如果找到答案的话就返回，否则再在左区间寻找。

```
int getnxt(int root, int left, int right, int num) {//寻找num的后继
  if (!tree[root]) return 0;//该区间为无后继
  if (left == right) return left;
  int lc = root << 1, rc = root << 1 | 1, mid = (left + right) >> 1;
  if (num >= mid) return getnxt(rc, mid + 1, right, num);//后继一定在右子树上
  int t = getnxt(lc, left, mid, num);//后继不一定在右子树上，尝试在左子树上找
  if (t) return t;//在左子树上找到
  return getnxt(rc, mid + 1, right, num);//在左子树上没有找到，再尝试在右子树上找
}
```

3 动态开点线段树

3.1 定义

在一些计数问题中，线段树用于维护值域（一段权值范围），这样的线段树也称为权值线段树。为了**降低空间复杂度**，我们可以**不建出整棵线段树的结构**，而是在最初只建立一个根节点，代表整个区间，当**需要访线段树的某棵子树（某个子区间）时，再建立代表这个子区间的节点。**采用这种方法维护的线段树称为动态开点的线段树。

动态开点的线段树抛弃了完全二叉树父子节点的 2 倍编号规则，改为使用变量记录左右子节点的编号（相当于指针）。

一般普通的线段树开局直接定义$4*N$的空间，然而当N很大时，4倍空间会消耗很多，这时考虑用动态开点线段树，**用多少开多少。不用的点不开，**可以大幅节省空间。空间复杂度只与修改次数有关，也就是$O(q\log N)$。

初始化：一个根节点`root`,存值数组`val[N]`,左右儿子节点编号`lc[N], rc[N]`

算法流程：

1.一开始只有一个根节点

2.**修改操作:**每次传入节点p,区间范围$[l,r]$和更新点x,判断x在区间的哪边，在哪边就就往哪边走，如果遇到一个还没有创建的节点但又需要往这个节点走，就以访问次序创建这个节点标号，像正常线段树更新即可。

3.**查询操作:**和一般线段树不同的是，当我们遇到一个没有访问过的节点，直接返回。其他与线段树查询一致。

3.2 操作

3.2.1 单点修改

```
void update(int &p, int l, int r, int x, int val) {
  if (!p) p = ++cnt; //遇到了，但没标号，标记
  if (l == r) {
    c[p] += val;
    return;
  }
  int mid = (l + r) >> 1;
  if (x <= mid) update(lc[p], l, mid, x, val);
  else update(rc[p], mid + 1, r, x, val);
  c[p] = c[lc[p]] + c[rc[p]];
}

```

3.2.2 区间查询

```
int query(int p, int l, int r, int x, int y) {
  if (!p) return 0; //p为0表示还未创建该节点
  if (x <= l && r <= y) return c[p];
  int mid = (l + r) >> 1, sum = 0;
  if (y <= mid) sum += query(lc[p], l, mid, x, y);
  if (x > mid) sum += query(rc[p], mid + 1, r, x, y);
  return sum;
}

```

3.3 复杂度分析

因为树上的任意一条链长度平均约为$\log 2N$，所以对于任意的操作，我们一直都是在一条长度 $d <= \log_2 N$ 的链上进行操作。

一棵维护值域$[1,N]$的动态开点的线段树在经历 m 次单点操作后，节点数量的规模为 $O(m\log_2 n)$，最终至多有 $2n-1$ 个节点。